

MotifVM: An Invariant-Checked Reasoning Runtime

Core Freeze at the Adapter Boundary

Amul Rangne

MotifVM Contributors

Version 0.5.4

Abstract

Large language model systems often blur the boundary between perception, reasoning, execution, and trust. MotifVM separates these concerns by treating model and tool outputs as proposed state transitions over an explicit cognitive state. This paper describes the v0.5.4 core freeze: artifacts enter through adapters, adapters emit hash-bound evidence references and normalized extracted facts, passes propose state patches, and the runtime validates, authorizes, applies, verifies, commits, and exports independently verifiable audit packs. The result is not another agent framework, but a small reasoning kernel with device-driver-like artifact adapters.

1 Thesis

MotifVM reframes the LLM reasoning problem. Instead of asking whether a model can reason over arbitrary messy inputs, MotifVM asks whether messy inputs can be converted into stable evidence and facts, and whether proposed reasoning transitions can be checked before they become state.

$$\text{Artifact} \rightarrow \text{EvidenceRef} \rightarrow \text{ExtractedFact} \rightarrow \text{Claim}$$
$$\text{CognitiveState} + \text{StatePatch} \rightarrow \text{validate} \rightarrow \text{authorize} \rightarrow \text{apply} \rightarrow \text{verify} \rightarrow \text{commit}$$

This division is the central design choice. Text and chaos remain at the boundary. Structure exists inside the virtual machine.

2 Kernel and Drivers

The MotifVM kernel contains the state model, patch semantics, permissions, invariants, terminal states, audit packs, and verifier. Artifact adapters are analogous to device drivers. They let the kernel speak to external artifact types without changing the kernel itself.

Kernel primitive	Role
CognitiveState	Explicit runtime memory
StatePatch	Only mutation vehicle
PatchAuthorization	Permission boundary
ArtifactAdapter	Artifact ingestion driver
EvidenceRef	Stable hash-bound locator
ExtractedFact	Normal form before claims
Invariant	Runtime correctness predicate
AuthorityRef	Source-bound rule reference
TerminalState	Committed success or failure
AuditPack	Portable evidence bundle
PackVerifier	Independent consistency checker
PatchTimeline	State-transition log
ReasoningGraph	Claim/evidence topology

3 Adapter Boundary

An artifact adapter has one job: convert an external artifact into evidence and facts. It may not mutate cognitive state. It may not emit terminal claims. It may not mark an invariant as passed. It is a perception layer, not a reasoning layer.

$$\text{ArtifactAdapter}(a) = (\text{EvidenceRef}^*, \text{ExtractedFact}^*)$$

Each **EvidenceRef** contains an input identifier, input hash, locator type, locator, and excerpt. Each **ExtractedFact** references one of those evidence records and carries a normalized value with confidence and extraction metadata.

4 ExtractedFact Normal Form

The v0.5.4 milestone makes **ExtractedFact** first-class. This is the bridge between raw evidence and domain claims. For example:

- A CSV row `tier1,120` becomes a `dcccb_component` fact.
- A diff line `return True` becomes a `code_added_line` fact.
- A changed repository file becomes a `repo_changed_file` fact.

Facts are deliberately weaker than claims. A fact says what was extracted. A claim says what the runtime concludes after domain logic, authority, and invariants.

5 Patch Semantics

No actor writes directly to **CognitiveState**. Passes, tools, adapters, and LLM calls can only propose **StatePatch** transitions. The runtime then checks schema validity, role authorization, graph consistency, domain invariants, and terminal semantics.

LLM and tool outputs do not become truth. They become proposed transitions.

This creates a clean trust boundary. A model may summarize. It may not silently change terminal status, invariant results, input manifests, authority references, or source artifacts.

6 Audit Packs

An audit pack is a portable evidence bundle. It contains the committed state, graph, lineage, invariants, input manifest, extracted facts, authority references, LLM calls, reconciliation patches, and patch timeline.

The verifier checks internal consistency without rerunning the task: input hashes, authority hashes, section hashes, graph endpoints, lineage references, reconciliation policy, patch timeline shape, and adapter-output conformance.

7 Results

By v0.5.4, MotifVM has passed the following release gates:

Version	Core proof	Evidence
0.1.8	Kernel and two-domain audit packs	Commit + audit exports
0.2.5	Provider boundary and evaluation	Structured calls + eval harness
0.3.5	Adversarial reliability	20-case adversarial suite
0.4.5	Scale and integration	Repository review + adversarial-100
0.5.4	Adapter boundary freeze	Adapter conformance + extracted facts

8 Limitations

MotifVM does not prove that every domain rule is complete, every regulation is current, or every security flaw is detected. It proves something narrower and more architectural: the runtime preserved the declared evidence, authority, permission, transition, invariant, and audit-pack boundaries for the task it ran.

9 Conclusion

MotifVM is best understood as an auditable cognitive virtual machine. Its core invention is not another orchestration graph or prompt template. Its core invention is the refusal to treat model and tool output as state. Output must pass through adapters, facts, patches, authorization, invariants, and committed terminal states.

With the adapter boundary frozen, future work becomes surface expansion: more adapters, more domain profiles, more invariants, better extraction, better user interfaces, larger evaluations, and real deployments. The kernel itself has a stable shape.